B

Academic Project Review 1

# Design and Implementation of a Database Management System for Bank Account and Transaction Management System

Project 55 · Bank Account & Transaction Management System


**Presented by**
M.Srilakshmi Reddy


**Course**
AI-ML

**Academic Year**
2026

# Problem Identification / Statement

### Existing Operational Issues

- Manual, inconsistent record-keeping across branches leads to balance errors.
- No single source of truth for joint account holders and linked accounts.
- Duplicate or clashing account / transaction IDs during peak processing.
- Transfers can be posted without a guaranteed debit-credit balance.
- Withdrawals sometimes processed against inactive or over-drawn accounts.
- Loan repayments tracked separately from the original repayment schedule.

### Problem Statement

*"Banks require a centralized, constraint-enforced relational database that keeps customer, account, transaction, transfer, and loan records consistent, auditable, and available in real time — without manual reconciliation."*

# Scope & Objectives

### Scope

- Conceptual (ER) and logical (relational) design covering all core banking entities.
- Normalization of the schema up to Third Normal Form (3NF).
- Implementation-ready schema with primary keys, foreign keys, and integrity constraints.
- Realistic sample data and SQL queries for reporting and auditing.
- A small front-end application demonstrating core banking operations.

*Out of scope: mobile apps, real-time payment gateways, multi-currency accounts, regulatory certification.*

### Objectives

1. Identify major entities, attributes, relationships, and user roles.
2. Produce a conceptual ER model and convert it into a normalized relational schema.
3. Enforce key business rules through constraints — unique IDs, balanced transfers, active-account checks, positive amounts.
4. Populate realistic sample data and validate it with SQL queries and reports.
5. Build a small front-end app demonstrating CRUD, search, and validation.
6. Demonstrate data integrity and indexing through systematic testing.

# Users & Functional Requirements

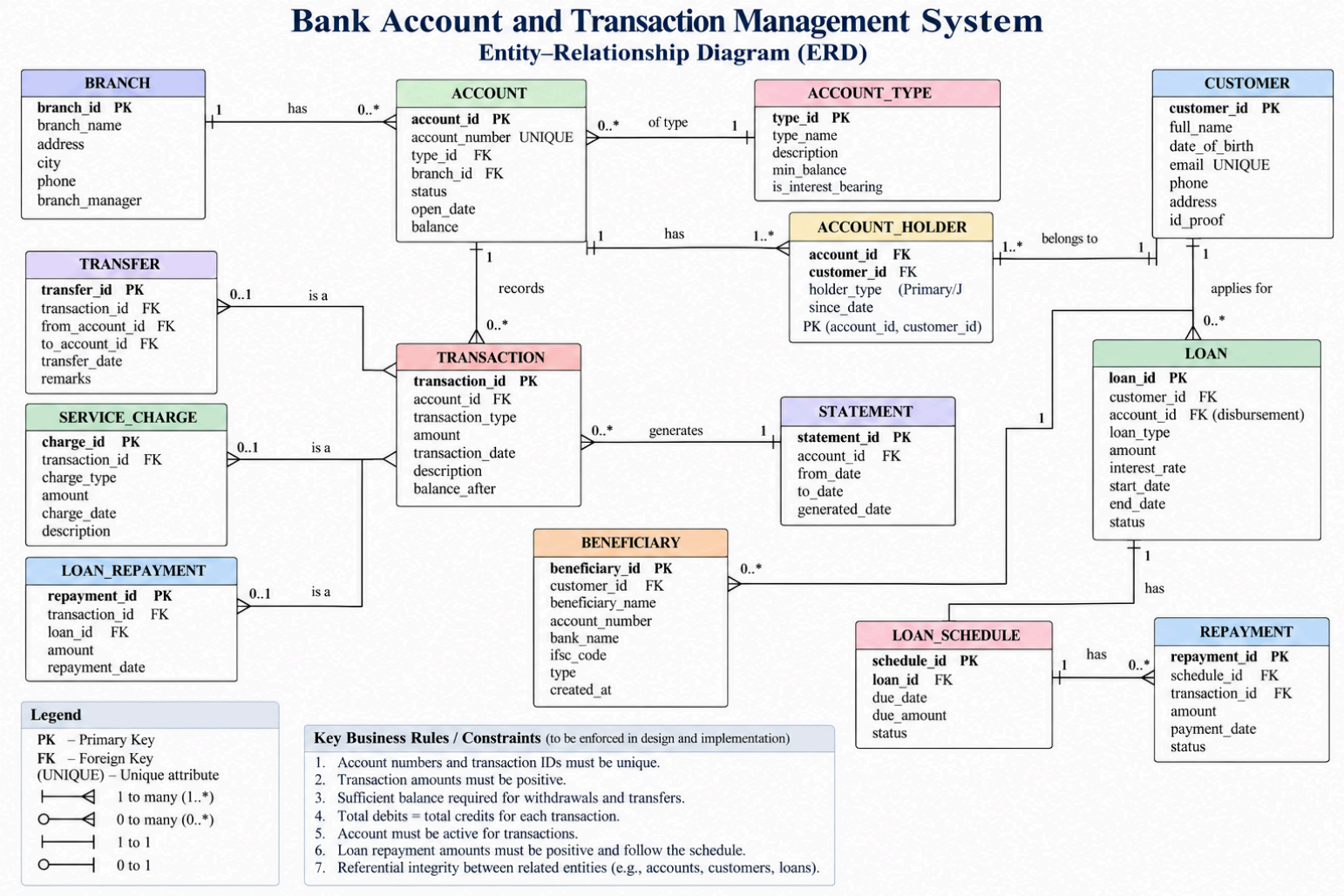
### Intended Users

- **Customer** — views balances, transactions, and statements.
- **Teller / Branch Staff** — manages accounts, deposits, withdrawals, and transfers.
- **Loan Officer** — originates loans and tracks repayment schedules.
- **Branch Manager** — reviews branch-level reports and summaries.
- **Database Administrator** — maintains schema integrity and access control.

### Functional Requirements

1. Register customer profiles and open accounts, including joint holdings.
2. Record deposits, withdrawals, and transfers with balanced debit/credit.
3. Maintain beneficiaries and apply service charges per account.
4. Originate loans, generate schedules, and record repayments.
5. Generate account statements and branch-wise reports.
6. Enforce unique IDs, positive amounts, and active-account rules.

# Conceptual Design / ER Diagram



# Initial Relational Schema

| Table | PK | FK |
|---|---|---|
| **BRANCH** | branch_id | — |
| **ACCOUNT_TYPE** | type_id | — |
| **CUSTOMER** | customer_id | — |
| **ACCOUNT** | account_id | type_id, branch_id |
| **ACCOUNT_HOLDER** | account_id, customer_id | account_id, customer_id |

| Table | PK | FK |
|---|---|---|
| **TRANSACTION** | transaction_id | account_id |
| **TRANSFER** | transfer_id | transaction_id, from/to acct. |
| **SERVICE_CHARGE** | charge_id | transaction_id |
| **LOAN_REPAYMENT** | repayment_id | transaction_id, loan_id |
| **STATEMENT** | statement_id | account_id |

| Table | PK | FK |
|---|---|---|
| **BENEFICIARY** | beneficiary_id | customer_id |
| **LOAN** | loan_id | customer_id, account_id |
| **LOAN_SCHEDULE** | schedule_id | loan_id |
| **REPAYMENT** | repayment_id | schedule_id, transaction_id |

# Thank You

Questions & Technical Discussion

Project 55 · Review 1